



Universidad Nacional Autónoma de México

Facultad de Ingeniería

División de Ingeniería Eléctrica

Departamento de Ingeniería en Computación

Cómputo Móvil (2026-2)

Grupo 03

Profesor:

Ing. Marduk Pérez de Lara Domínguez

Trabajo Final - BocadoApp

Equipo BitLab:

Cervantes García Eduardo

Márquez Sánchez Mirna Daniela

Pilares Garcia Heber Adan

Fecha de entrega 29/05/2026

Somos BitLab, nuestro core es ofrecer soluciones tecnológicas centradas en las personas, nuestra visión es crear herramientas digitales que no solamente resuelvan problemas técnicos, sino que también generen un impacto real en la sociedad. Creemos que la tecnología y hoy en día la inteligencia artificial deben utilizarse para acercar a soluciones más accesibles y sostenibles, específicamente en las áreas donde hay demasiada complejidad o desinformación.

Así mismo, buscamos combinar innovación, UX y el análisis inteligente de datos para así construir productos que respondan a necesidades reales de la sociedad.

De ahí, nace BocadoApp, una propuesta innovadora en el sector de la salud digital con un objetivo humano muy claro: transformar por completo la relación que las personas tienen con la comida. El propósito central al generar esta aplicación es cambiar la conversación actual en torno a la alimentación; se busca erradicar el miedo, la culpa y la ansiedad que suelen provocar las dietas extremas o el conteo estricto de calorías, bajo la firme convicción de que comer bien se tiene que sentir bien. No se diseñó con la intención de reemplazar a los profesionales de la nutrición, sino todo lo contrario: busca ser una herramienta de apoyo y un puente directo de conexión para que los usuarios puedan llevar un mejor seguimiento de la mano de sus nutriólogos.

El desarrollo de Bocado fue posible gracias al trabajo colaborativo de BitLab, integrando distintas áreas relacionadas con desarrollo móvil, experiencia de usuario, inteligencia artificial y gestión tecnológica. Teniendo roles estratégicos destinados a funciones específicas que ayudan a la construcción completa de cualquier idea innovadora.

- Mirna — Project Manager (PM) y UX/UI Designer
- Eduardo — Backend Developer y Database Manager
- Heber — AI Engineer y iOS Developer

En el equipo de trabajo, se sigue un enfoque 100% colaborativo e iterativo, tenemos como base principios de metodología ágiles para que así sea más fácil la comunicación en equipo, la adaptación a cambios y priorización de funcionalidades.

Se inició por una etapa de investigación y análisis del problema, donde el equipo identificó las principales problemáticas relacionadas con hábitos alimenticios, obesidad, diabetes y desinformación nutricional. A partir de esto, se definió el objetivo principal de Bocado y se establecieron las funcionalidades prioritarias que formarían parte del flujo principal de la aplicación.

Posteriormente, el equipo organizó el trabajo mediante la división de roles previamente mencionados y asignación de tareas específicas. Cada integrante participó tanto en la parte técnica como en el análisis funcional de la solución, permitiendo una colaboración multidisciplinaria.

Durante el desarrollo se utilizaron herramientas de administración y colaboración para mantener seguimiento de tareas, comunicación y documentación del proyecto. Entre las principales herramientas utilizadas se encuentran:

- Figma, para diseño de interfaces, wireframes y prototipos visuales.

- GitHub, para control de versiones y organización del código fuente.
- Notion y herramientas colaborativas, para planeación de tareas, documentación y seguimiento del avance.
- Google Drive y KeyNote, para desarrollo de documentación, presentaciones y organización de recursos visuales.

BitLab se enfocó en crear una experiencia sumamente sencilla, intuitiva y libre de restricciones. Con el lema "Comer bien, debe ser más simple", la aplicación apuesta por un diseño minimalista donde "menos es más". Lo que se busca es que usar la app sea algo tan natural como enviar un mensaje de texto, integrándose sin esfuerzo en la rutina diaria de estudiantes, profesionistas y cualquier persona que desee construir hábitos saludables a largo plazo.

Para lograrlo, la aplicación organiza su propuesta de valor en cuatro pantallas clave que fueron el punto de inicio y de mayor esfuerzo en detalle.

- **El Escáner Principal:** Inspirado en el funcionamiento inmediato de aplicaciones como Shazam, permite que el usuario tome una foto de su plato y reciba en segundos un análisis de sus alimentos, acompañado de sugerencias y advertencias amigables en lugar de prohibiciones.
- **Mi Diario:** Un espacio diseñado para que el usuario registre sus comidas diarias y, lo más importante, aprenda progresivamente de ellas para mejorar de forma autónoma sus hábitos.
- **Aprende:** Una sección dedicada a la educación alimentaria donde los nutriólogos pueden difundir sus investigaciones, permitiendo que la gente se informe con fuentes confiables y pueda contactar directamente a los especialistas.
- **Perfil:** El rincón personalizado donde el usuario gestiona sus metas individuales (como ganar masa muscular), registra sus datos y mantiene enlazada la cuenta con su nutriólogo de cabecera.



Imagen 1.1 Pantalla principal

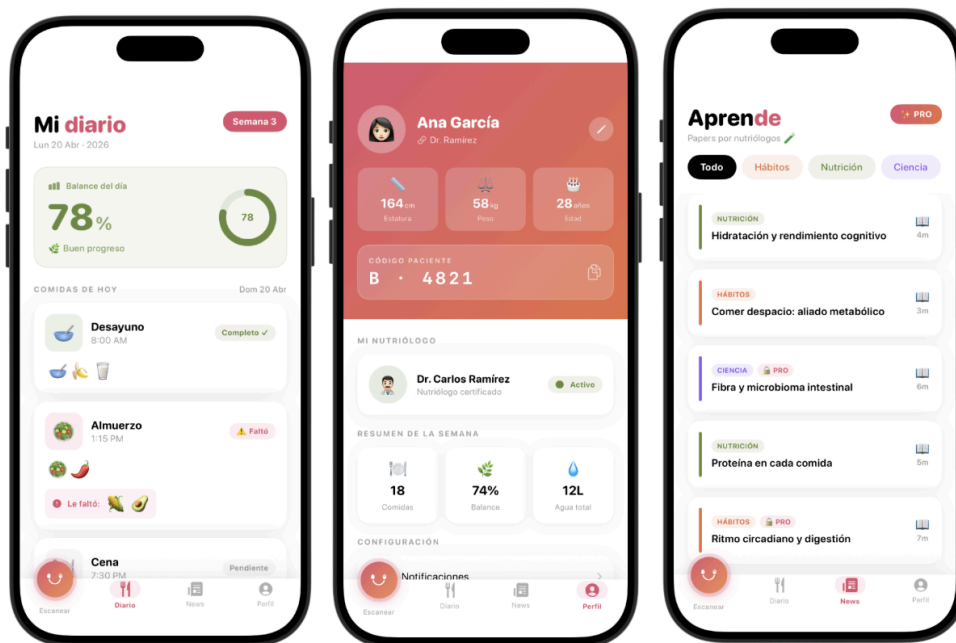


Imagen 1.2 Pantallas secundarias

El alcance definido para el Producto Mínimo Viable, se limitó de manera estricta a demostrar la viabilidad técnica del motor analítico de BocadoApp en el ecosistema iOS, priorizando exclusivamente el flujo central de captura fotográfica, la conversión de la imagen a Base64 y la obtención de datos a través de la API externa de Gemini. Al operar bajo la extrema presión del evento, el equipo se vio obligado a centrar sus esfuerzos en la pantalla de escaneo tipo Shazam, dejando de lado la conectividad con dispositivos de salud externos y asumiendo una deuda técnica considerable debido a la velocidad de ejecución y la falta de madurez de la base de datos.

Fuera del entorno competitivo del hackathon, el alcance se reconfigura para estructurar una primera versión realista de la aplicación apta para su lanzamiento en el mercado mexicano, consolidando los módulos secundarios de la plataforma que dan soporte al modelo de negocio híbrido. Esta primera entrega comercial formaliza la sección "Mi Diario" para permitir el almacenamiento histórico de las comidas, el conteo interactivo de agua y el seguimiento de metas, además de habilitar el apartado educativo "Aprende" donde los nutriólogos pueden publicar sus investigaciones y vincularse con pacientes mediante un número de identificación único.

Para mantener este primer lanzamiento dentro de parámetros realistas y estables, se postergan para fases futuras el desarrollo de modelos de Machine Learning locales basados en visión por computadora en el hardware del iPhone y la integración con tecnologías o variables biométricas en tiempo real. De este modo, la primera versión oficial se enfoca en ofrecer la experiencia del escaneo gratuito asistido por anuncios no invasivos y la suite de suscripción para los profesionales de la salud, manteniendo el software exclusivamente dentro de los estándares de SwiftUI y el sistema operativo de Apple para garantizar un rendimiento óptimo y seguro frente al mercado.

Una experiencia única, Bocado inicia con una pantalla de bienvenida donde se presenta la identidad de la app y se introduce al usuario al concepto principal: mejorar su alimentación mediante el análisis de sus comidas. Aquí se agrega un disclaimer para que el usuario sepa que esta app tiene la ayuda de una IA, que recomendamos el seguimiento y acompañamiento de un profesional de la salud, somos una herramienta, un brazo extendido, NO queremos reemplazar a este personal clave. Desde esta primera pantalla, el usuario avanza hacia la pantalla principal de escaneo.

La funcionalidad principal de la aplicación se concentra en el botón central de escaneo, representado como un botón grande e intuitivo que invita al usuario a tomar una foto de su comida. Este botón es el eje de interacción más importante, ya que activa el proceso completo de análisis nutricional mediante inteligencia artificial.

Una vez que el usuario toca el botón de escaneo, la aplicación abre la cámara del dispositivo. En esta pantalla, el usuario puede enfocar su comida y presionar el botón de captura. Después de tomar la fotografía, la app cambia a un estado de análisis, donde se muestra una pantalla de espera indicando que la inteligencia artificial está revisando el plato.

Cuando termina el procesamiento, la aplicación presenta una pantalla de resultados con información nutricional del alimento detectado, incluyendo calorías, proteínas, carbohidratos, grasas, fibra, sodio y azúcar. Además, muestra recomendaciones y advertencias relacionadas con el consumo del alimento. Desde esta pantalla, el usuario puede guardar el resultado en su diario.

Al guardar el análisis, la información se integra al módulo Mi diario, donde el usuario puede consultar su balance diario, las comidas registradas y el avance de sus hábitos alimenticios. Desde el menú inferior también se puede acceder al perfil del usuario, al perfil del nutriólogo, a la pantalla premium de Plan Plus, al contenido educativo y al módulo de ejercicios.

El recorrido general de las pantallas puede entenderse así:

Bienvenida → Escaneo → Cámara → Análisis → Resultado nutricional → Guardar en diario → Diario del usuario

A partir de ese flujo principal, el usuario puede navegar hacia pantallas complementarias como:

Perfil del paciente → Perfil del nutriólogo → Plan Plus → Noticias / aprendizaje → Ejercicios semanales

El flujo de la app está diseñado alrededor de una acción principal: tomar una foto de la comida. Por eso, el botón gigante de escaneo es el centro de la experiencia. La idea es que el usuario no tenga que llenar formularios complicados, sino simplemente capturar su plato y dejar que la inteligencia artificial haga el análisis.

Una vez teniendo claro las funcionalidades clave que buscábamos que cumpliera Bocado, pasamos al boceto de estas ideas, lo que se le conoce como wireframe, para así distribuir cómo estaría el contenido y más importante cómo sería la navegación del usuario. Así pasar de la idea a una realidad.

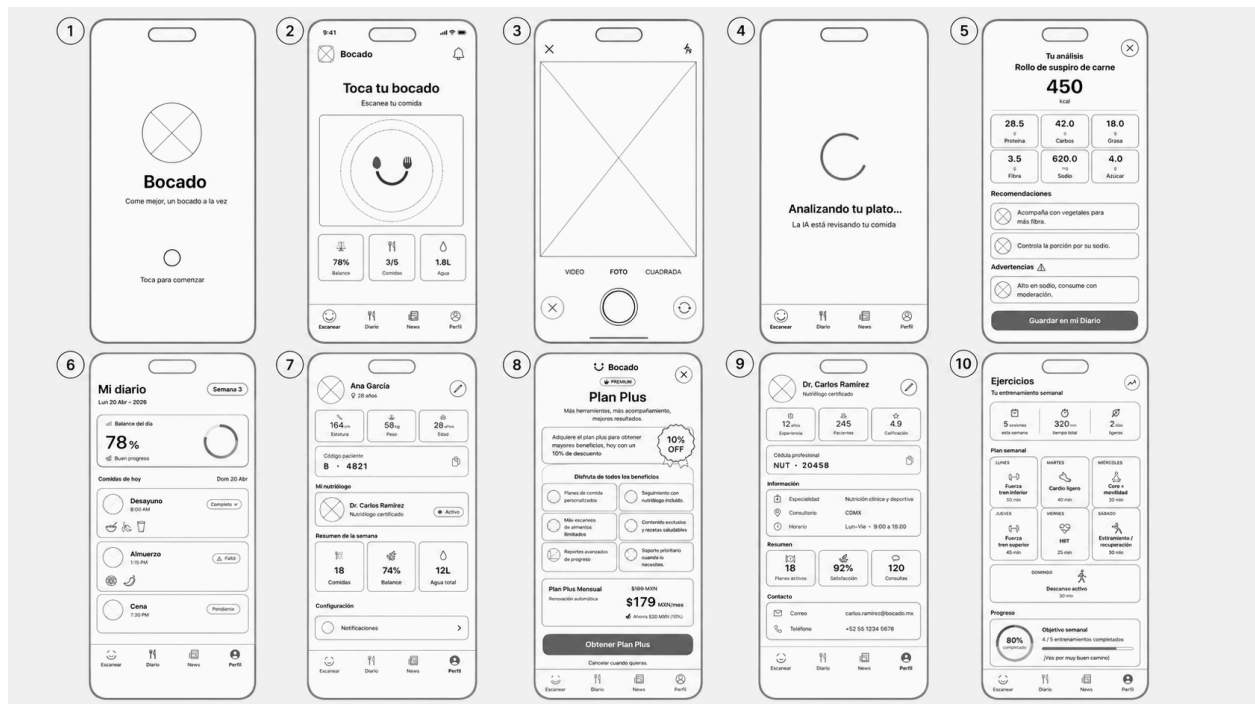


Imagen 1.3 Wireframes de Bocado App



Imagen 1.4 Funcionalidades y pantallas por segmentos (1)

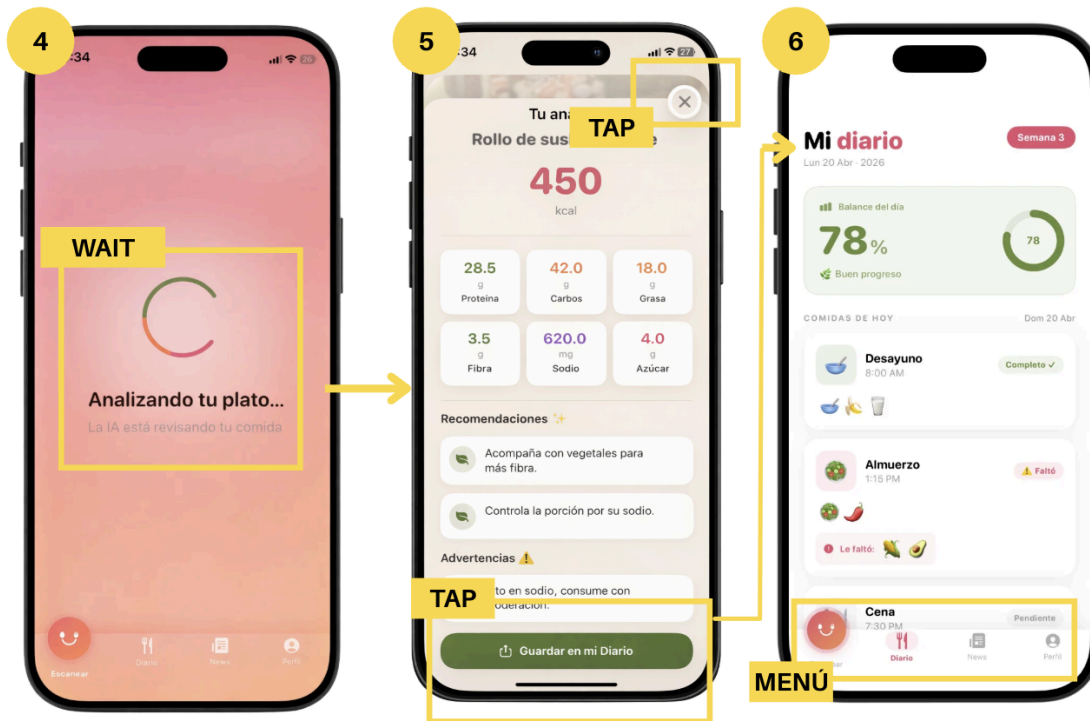


Imagen 1.5 Funcionalidades y pantallas por segmentos (2)



Imagen 1.6 Funcionalidades y pantallas por segmentos (3)

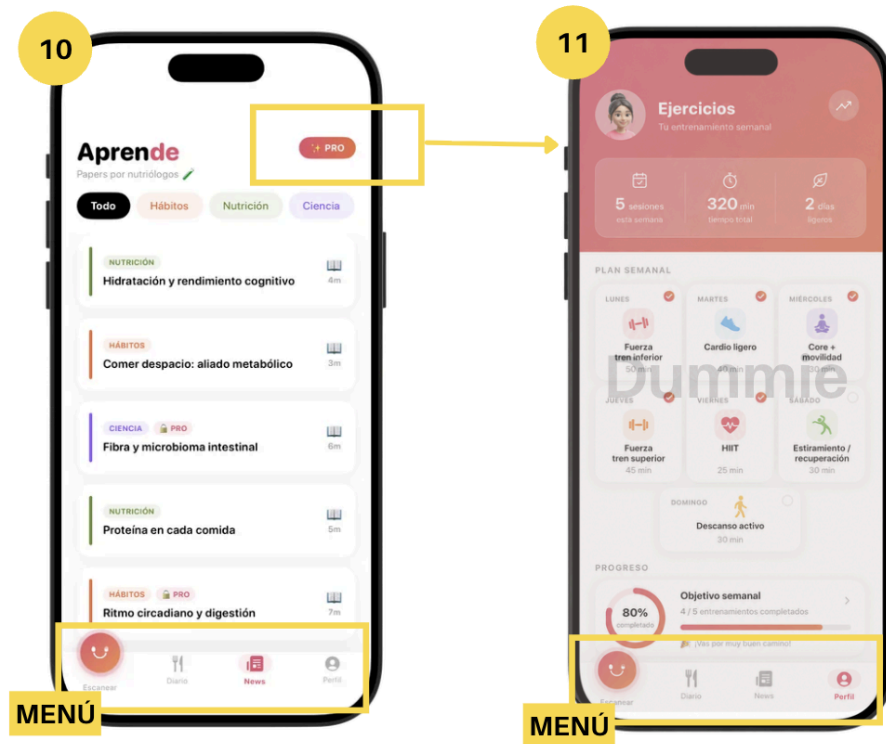


Imagen 1.7 Funcionalidades y pantallas por segmentos (4)

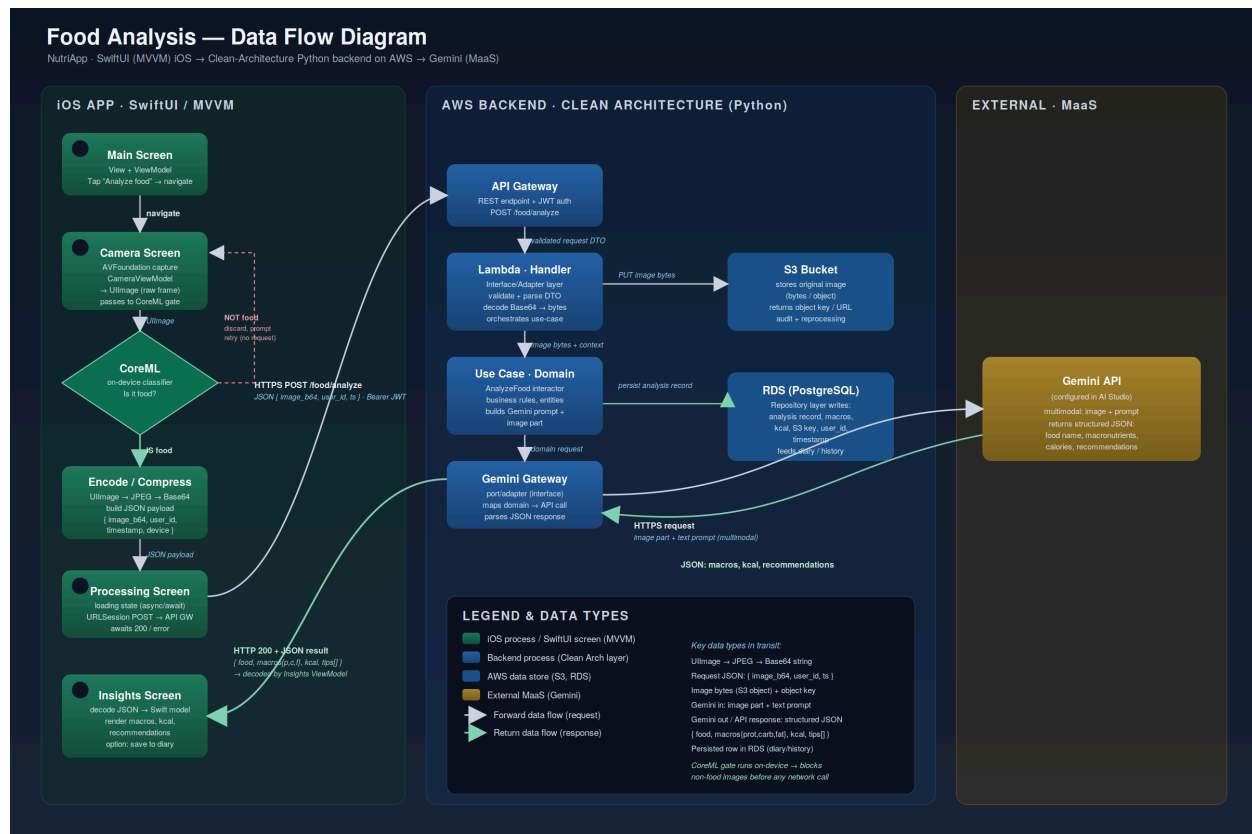


Imagen 1.8 Data Flow Diagram (AI)

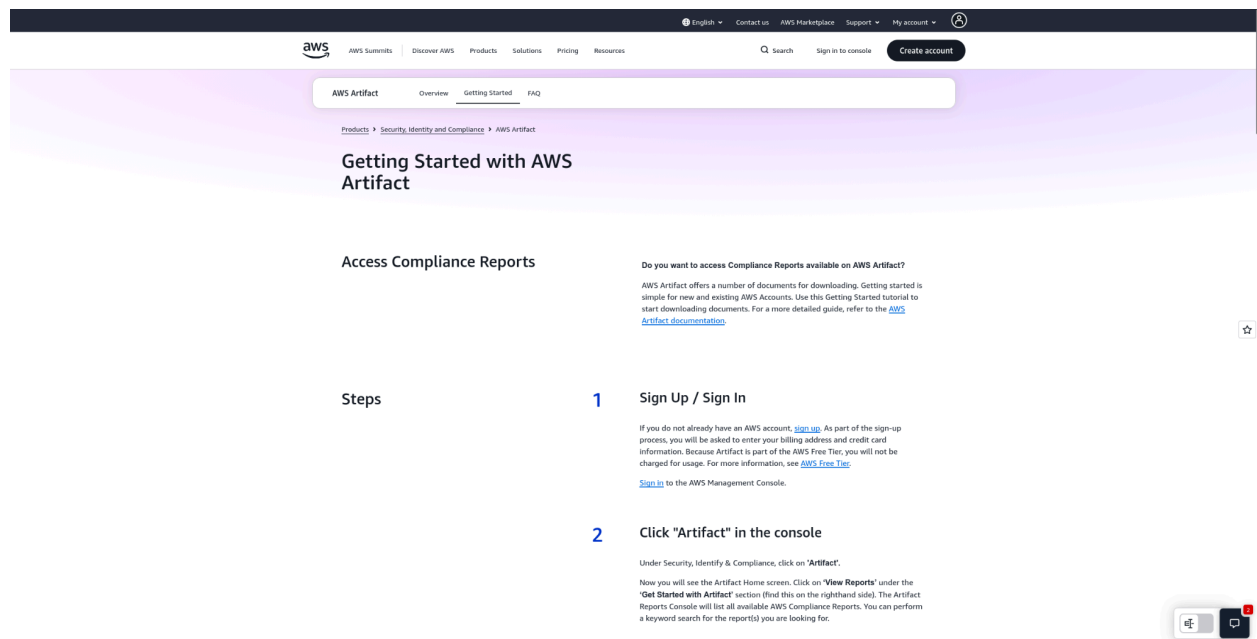
De cara al usuario final este servicio será un SaaS (Software as a Service), para Bitlab será un PaaS (Platform as a Service). Esto implica que todos los datos registrados de clientes mediante la aplicación móvil, se guardarán en la instancia de base de datos que se tendrá, por ejemplo en AWS, y sólo datos de acceso constante como un plan actual de alimentación o el diario de la aplicación se guardarán en almacenamiento del teléfono. La decisión de esta modalidad de almacenamiento de datos reside en dos factores claves para Bitlab, escalabilidad y seguridad.

Es bien reconocido que las aplicaciones SaaS se vuelven accesibles para cualquier cliente con acceso a internet, ya que en muchos casos pueden ejecutarse desde un navegador web. Bitlab no adoptará la opción de una aplicación web, sino una móvil, pero esta accesibilidad que la computación como servicio da + los clientes permitirá una adopción rápida. Los clientes solo descargarán Bocado y tienen acceso al menos a todas las funciones gratis de inmediato.

El modelo SaaS hacía el cliente y PaaS hacía Bitlab también añade valor en cuestión de seguridad, al ser una startup no se tiene el personal, ni la infraestructura para guardar información personal relacionada a la salud de forma segura. Haciendo uso de infraestructura de grandes proveedores en la nube y de un modelo de venta SaaS, se

puede estar respaldado con un reporte SOC2 Type 2 para la infraestructura que almacena los datos, es algo que todos los grandes centros de cómputo en la nube suelen tener.

Un SOC2 Type 2 es una certificación que garantiza que el proveedor de servicios en la nube cuenta con principios básicos en seguridad que garantizan la tríada básica de la ciberseguridad: confidencialidad, integridad y disponibilidad. Los criterios de esta certificación pueden extenderse también por ejemplo a privacidad, pero al menos debe estar garantizada la tríada. Esta certificación se renueva cada año y el tipo 2 en su nombre hace referencia a que la validación implica una auditoría prolongada que usualmente dura 6 meses. Proveedores como AWS tienen sitios de transparencia donde mencionan todas las certificaciones con las que cuentan y brindan modos de acceder o visualizar los reportes.



Pantalla de AWS Artifact[3]

Además de ofrecer el software como un servicio al cliente, de acuerdo al modelo de negocio establecido y las posibilidades de Bitlab, cómo ya se mencionó se optó por utilizar un modelo PaaS para alojar los servicios relacionados con el backend, en lugar de un modelo On-Premise. El principal factor para escoger un modelo PaaS frente a un IaaS u On-premise, es la escalabilidad, flexibilidad y sencillez que este modelo ofrece.

Inicialmente Bitlab es un equipo pequeño por lo que se prefiere delegar todo lo involucrado a infraestructura sin perder un entorno de desarrollo configurable a las necesidades del desarrollo. A diferencia de IaaS, PaaS ofrece sistemas operativos administrados, middleware, entornos en tiempo de ejecución y herramientas más avanzadas para implementar por ejemplo, serverless. Esta escalabilidad podría permitirnos por ejemplo adoptar una arquitectura serverless de microservicios que se

adapte a la demanda de clientes que Bocado tenga al momento. Es la mejor opción en cuestión económica y que permite estar un paso adelante de las necesidades de los clientes al mismo tiempo.

En los últimos párrafos se ha mencionado constantemente AWS (Amazon Web Services), ya que es la plataforma de servicios en la nube que se decidió usar porque cumple con todas las características que Bitlab necesita para Bocado. En primer lugar cuenta con alta disponibilidad alrededor de todo el mundo y sobre todo en el territorio inicial donde se desplegará la app, México, con la Región de AWS México (Central). En segundo lugar, al ser un proveedor grande de servicios en la nube garantiza SOC2, escalabilidad asegurada y ofrece servicios avanzados para desarrolladores. Un punto clave es que aunque el cómputo distribuido cuenta con conceptos universales o similares entre proveedores, Bitlab conoce las tecnologías de Amazon, esto permitirá que la infraestructura en la nube no sea un problema en el desarrollo y se pueda alcanzar la arquitectura deseada.

De los servicios de AWS se contratará RDS con un motor de base de datos open-source para la base de datos de la aplicación, un bucket S3 para contenido BLOB y para ofrecer los servicios del backend se planea usar Lambda, es decir, serverless.

En cuestión de Inteligencia Artificial, una característica presentada como central de la aplicación, se planea usar un modelo MaaS (Machine Learning as a Service). Esto brinda la capacidad de IA a la aplicación únicamente haciendo uso de un API; libera a la startup de entrenar un gran modelo de lenguaje o multimodal desde cero, a la vez permite integrar esta tecnología de manera sencilla y estar a la vanguardia con sus avances. Todo esto sin costear todos los recursos humanos y financieros que implica el desarrollo de una IA para un laboratorio de inteligencia artificial.

También debe notarse que se considera el uso de las capacidades de procesamiento de las NPU de los teléfonos modernos, esto con el fin de tener modelos ligeros en local, que por ejemplo eviten que se envíen a la API fotos que no son comida o que el modelo ligero envíe a la API un pre-análisis de la comida que permita un uso eficiente de tokens y ahorro económico con el modelo MaaS.

Con el modelo de servicio y tipo de hosting, ahora se debe ahondar en el ecosistema y dispositivos para los que esta aplicación estará disponible, se desarrollará inicialmente de manera única para el ecosistema Apple, específicamente para iPhones. La razón detrás de esta decisión es que no se tiene el equipo suficiente para abarcar un lanzamiento óptimo en plataformas más abiertas como android; y a la vez, gran parte del público objetivo de Bitlab está entre los usuarios de Iphone, ya que es un ecosistema en el que está comprobado que los usuarios suelen pagar más frecuentemente por aplicaciones y servicios. [9] [10]

Por esta razón el sistema mínimo para el que se desarrollará será el Iphone X, al tener casi una década de antigüedad se abarca una gran cantidad de dispositivos y a la vez fue el primer Iphone a partir de iOS 11 en implementar "Core ML" [7] mediante el uso

de NPU, lo cual es una característica deseada para los planes de desarrollo que se tiene de la aplicación con modelos locales de ML. Esto quiere decir que las especificaciones del sistema mínimo para poder instalar y ejecutar correctamente la aplicación Bocado son: [8][11]

- **Pantalla:** Resolución de 2436 x 1125 pixels a 458 ppi.
- **SoC:** Chip A11 Bionic con arquitectura de 64 bits en adelante.
 - 6 núcleos de CPU.
 - 2x 2.39 GHz – Monsoon
 - 4x 1.42 GHz – Mistral
 - Apple A11 Bionic GPU 1066 MHz
 - Apple Neural Engine
 - 4 GB RAM
- **Cámara:** Dispositivos con al menos una cámara de 12 MP.
- **Conectividad:**
 - Wi-Fi 802.11ac con MIMO
 - Tecnología inalámbrica Bluetooth 5.0
 - NFC con modo lectura
- **Sistema Operativo:** iOS 11 en adelante.



[8]

Como ya se mencionó estas características aseguran abarcar dispositivos de hasta un década de antigüedad sin sacrificar la innovación y planes de desarrollo que se tienen. Los 4GB de RAM es un estándar que se extendió por múltiples generaciones de Apple y que funciona bien con el sistema operativo, lo que a la vez nos permite enfocarnos en usar los recursos del dispositivo de forma eficiente. El cliente móvil será ligero, debido a que la mayoría del procesamiento se realizará en el backend soportado como PaaS, pero el cliente tendrá efectos avanzados en animaciones y gráficos lo cual ya era

posible y probado desde la A11 Bionic GPU 1066 MHz. Y cualquier implementación con el uso de la NPU ya es posible desde este dispositivo, según sus características, lo cual concuerda con el plan de usar modelos locales y ligeros de Machine Learning que hagan más eficiente el uso del MaaS y mejoren la experiencia de los usuarios.

Cambiando al software de la aplicación, para escoger la arquitectura de Bocado, se decidió usar principios Solid, que aseguren un código limpio, mantenible, orden, eficiencia, escalabilidad y adaptabilidad a nuevas tecnologías. Como parte de los principios Solid, se es consciente de que cada componente debe tener solo una responsabilidad (Single responsibility principle) y es por eso que la arquitectura de Bocado, a pesar de poder verse como un “Cliente-Servidor” a gran escala, consiste de una aplicación móvil con su propia arquitectura y un backend con la suya, que en conjunto permitirán ofrecer la experiencia que se espera al usuario.

Este patrón permite separar la lógica de interfaz de la lógica de la aplicación, es un patrón creado por Microsoft para resolver el exceso de llamadas del patrón MVP y actualmente es el estándar en aplicaciones móviles y páginas web. Se ha vuelto el estándar debido a que el patrón MVVM trabaja con “estado” lo que permite que la aplicación sea reactiva, lo cual se traduce a un mejor manejo de responsabilidades y una interfaz con sensación responsiva para el usuario. [13]

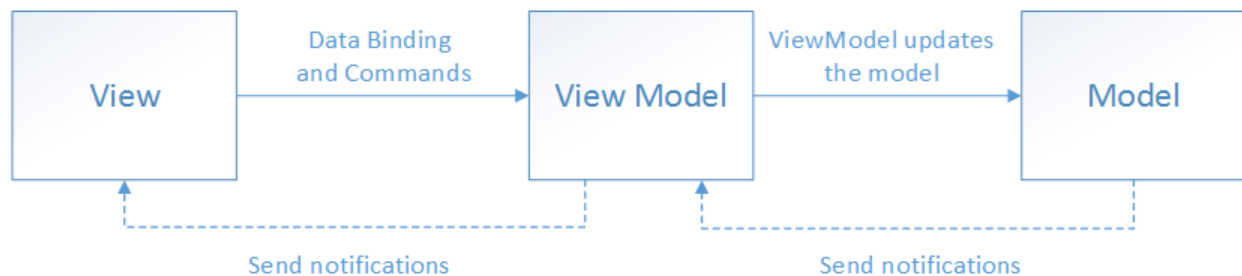
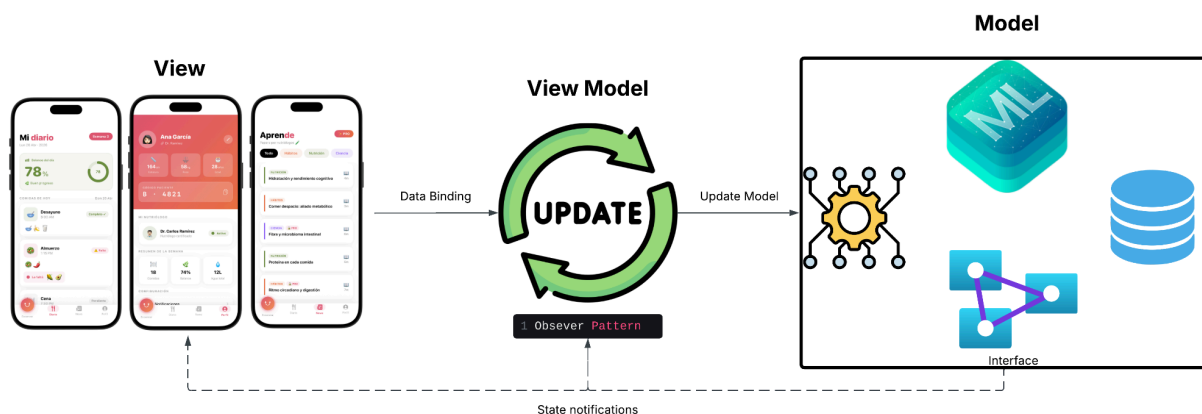
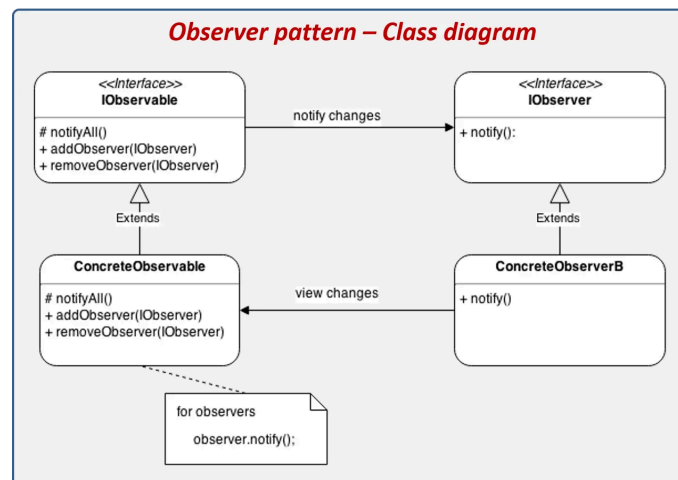


Diagrama oficial de Microsoft [13]



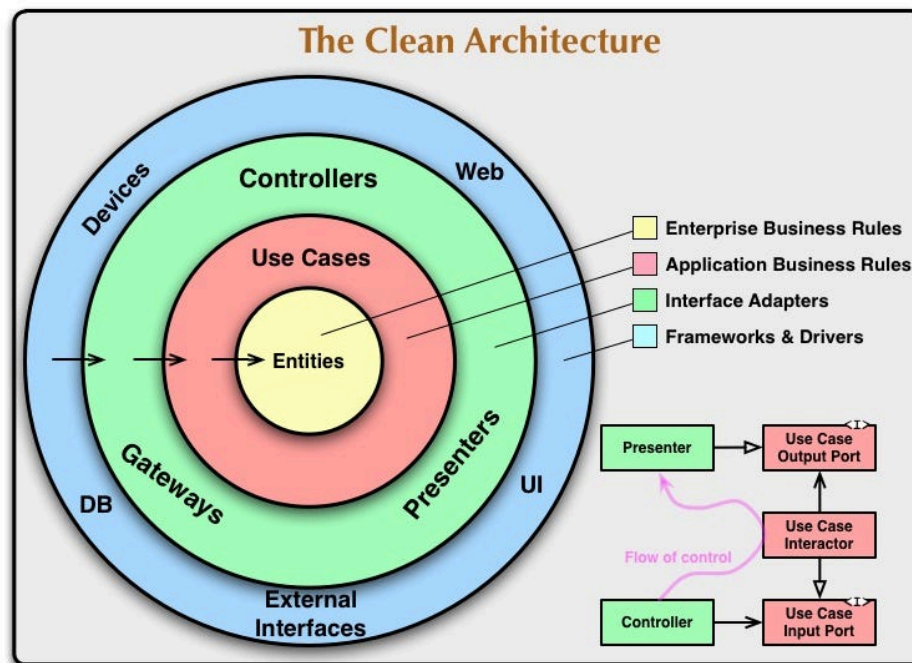
El diagrama presentado sería el diagrama de arquitectura móvil o presentación de bocado, donde en la View se tiene la interfaz apoyada en los componentes y framework que proporciona SwiftUI; en ViewModel se tendrá una clase que implementará el patrón observador para poder notificar el estado tanto a la interfaz o al Modelo, generando la reactividad de la aplicación; por último, en Model es donde se encontraría la lógica de la aplicación móvil, las interfaces (APIs) con el backend y algunas de las funcionalidades que se delegarán al hardware del iPhone como el uso de CoreML para mejorar la eficiencia de funcionamiento así como la económica en cuanto al costo de tokens de la API de IA.

Como se mencionó, uno de los patrones de diseño que se aplicará a nivel de código, es el patrón observer. Se menciona este patrón ya que hace un sinergia especial con la arquitectura escogida para la aplicación Móvil. MVVM trabaja bajo el núcleo del estado y las notificaciones de este, y es ahí donde el patrón observador brilla, ya que “Observador permite observar los cambios producidos por un objeto, de esta forma, cada cambio que afecte el estado del objeto observado lanzará una notificación a los observadores; a esto se le conoce como Publicador-Suscriptor”. [14]



Patrón de diseño observer para los estados de la aplicación [14]

Diagram de arquitectura del backend (Clean architecture)



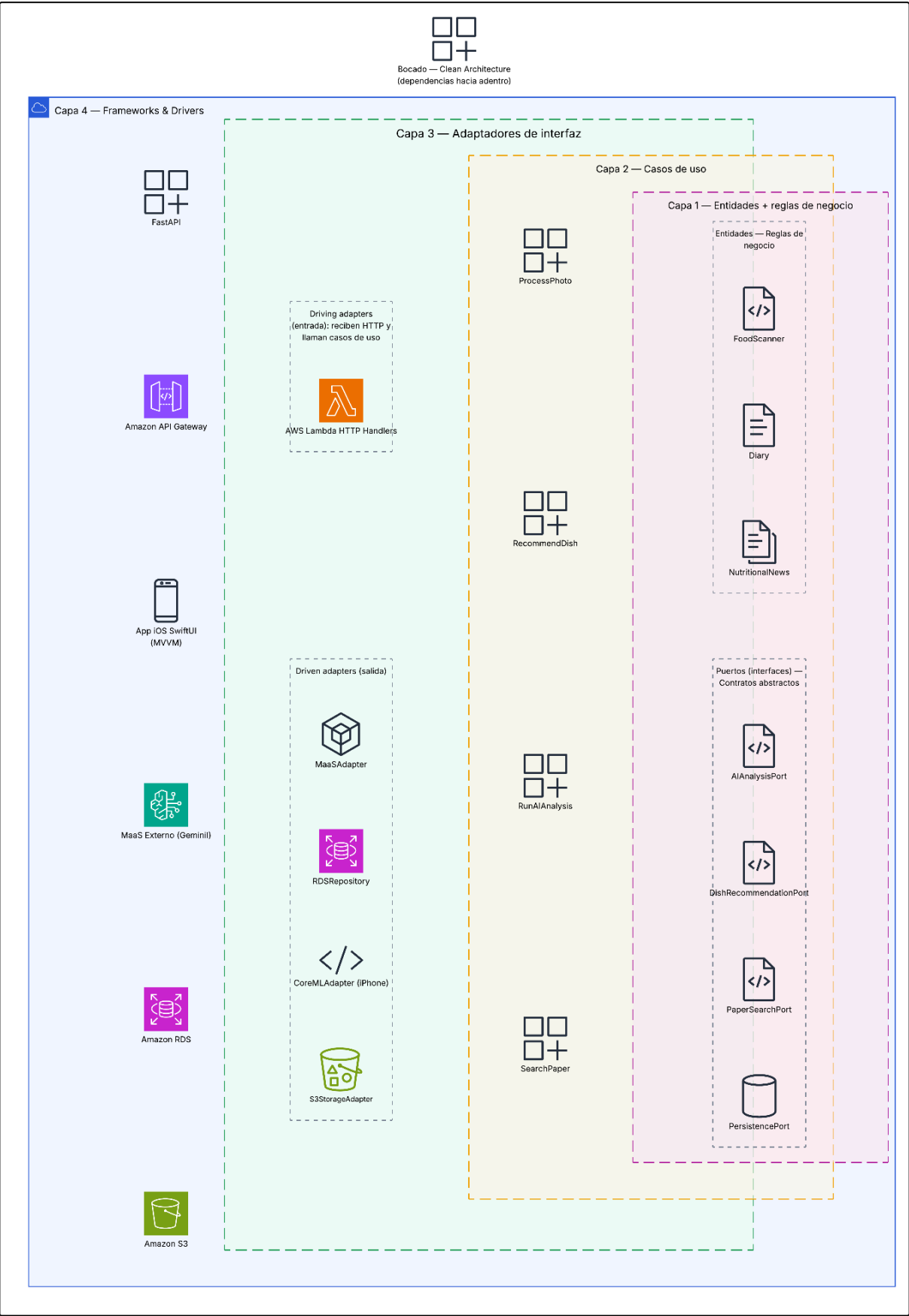
Medium Clean Architecture [12]

Para el backend o la funcionalidad principal del servicio que se ofrecerá a los usuarios, se optó por “Clean Architecture”. El core de Bocado está claro para el equipo, si bien está establecido en las reglas de negocio al inicio del documento, la aplicación consiste en un escaner de alimentos, diario, aprendizaje y el perfil del usuario; el alcance de cada regla puede expandirse, pero no cambiará ya que es lo que aporta valor a la aplicación. Lo mencionado anteriormente es lo que hace que esta arquitectura encaje perfectamente en el caso de uso.

“La idea fundamental detrás de Clean Architecture, es que la estructura y diseño de un software deben estar enfocados en separar las preocupaciones del negocio de las preocupaciones técnicas. El objetivo es que el software sea fácil de entender, mantener y evolucionar. En otras palabras, la Arquitectura Limpia busca crear un software de alta calidad, que sea sencillo de cambiar; y que además se adapte fácilmente a las necesidades mutables del negocio” [16]

Para Bocado se tiene el siguiente diagrama de arquitectura usando clean architecture, donde casi todos los componentes de las capas son reemplazables, exceptuando la capa 1, que sería una capa “estática”.

Bocado — Clean Architecture



Regresando a los principios Solid, esta arquitectura permite la división de responsabilidades y en un área tan cambiante actualmente como la de la inteligencia artificial, esto permite que podamos actualizar fácilmente a la vanguardia de los modelos. Hay otros ejemplos de beneficios de esta arquitectura, como un cambio sencillo de tipo de hosting o de framework como fastAPI por flask. El principio que se tiene en Bocado es que la idea que crea esta aplicación no se vea afectada por el avance de la tecnología ni por cómo pida el mercado que la aplicación cambie.

Entrando a las tecnologías de las que se tiene un vistazo en los diagramas, para la construcción de la aplicación móvil, el desarrollo se limitará exclusivamente al ecosistema iOS de Apple. El lenguaje de programación seleccionado es Swift, utilizando SwiftUI como framework principal para el diseño y gestión de la interfaz de usuario. Esta decisión se sustenta en que SwiftUI ofrece un enfoque declarativo moderno que optimiza el tiempo de desarrollo en comparación con el antiguo esquema de Objective-C o UIKit, facilitando la creación de interfaces reactivas y asíncronas bajo el patrón de arquitectura MVVM (Model-View-ViewModel). Además, desarrollar de manera nativa para iOS mitiga la inmensa fragmentación de dispositivos presente en otros sistemas operativos, garantizando un rendimiento eficiente y fluido del escáner en los modelos de iPhone.

Para el funcionamiento técnico del prototipo, se implementan bibliotecas nativas de Apple. El framework AVFoundation es el encargado de interactuar de forma directa con el hardware del dispositivo para solicitar los permisos de la cámara trasera y gestionar la captura de imágenes en alta resolución. Una vez capturada la fotografía, se emplean las herramientas de preprocesamiento nativas de Swift para redimensionar y optimizar la imagen, compartiéndola a un flujo de datos en formato JPEG. Posteriormente, se utiliza la biblioteca estándar del sistema para realizar la codificación de este flujo binario en una cadena de texto en formato Base64, el cual es el estándar requerido para la transferencia de datos hacia los servicios en la nube MaaS.

El backend del sistema se desarrollará utilizando el lenguaje de programación Python, implementando FastAPI como framework principal para la creación de la API. La elección de FastAPI se justifica por su alto rendimiento, su soporte nativo para programación asíncrona (esencial para manejar peticiones en tiempo real que involucran procesamiento de imágenes) y su capacidad para generar documentación automática y validación de datos estructurales a través de Pydantic. El servidor actuará como una capa intermedia o *gateway* de comunicación asíncrona; se encargará de recibir las solicitudes POST provenientes de la aplicación iOS con la imagen en Base64, validar la estructura de la petición, adjuntar de forma segura los parámetros de autenticación y coordinar la comunicación con los servicios externos de Inteligencia Artificial.

La arquitectura de inteligencia artificial de BocadoApp está diseñada bajo un enfoque híbrido que combina la potencia de la nube con la eficiencia del procesamiento local en el dispositivo. En la primera etapa de desarrollo, se utiliza un modelo de Machine Learning como Servicio (MaaS) mediante la integración de la API multimodal Gemini 3.0 Flash de Google. La decisión de utilizar este servicio se justifica por su alta velocidad de procesamiento, su capacidad nativa para realizar razonamiento cruzado entre imágenes y texto, y su facilidad de implementación para alcanzar un Producto Mínimo Viable (MVP). La configuración del acceso a esta API se gestiona desde el entorno de Google AI Studio, donde se realizará un riguroso trabajo de ingeniería de prompts (*prompt engineering*). A través del establecimiento de un *System Prompt* (prompt de sistema) robusto, se entrena y delimita el comportamiento del modelo para que actúe estrictamente como un experto en nutrición, prohibiendo alucinaciones de datos y forzando de manera estricta que la salida sea entregada en un formato JSON estructurado que el backend pueda parsear de forma limpia y transparente para el cliente.

A futuro, el plan de escalabilidad contempla mitigar la dependencia absoluta de las APIs en la nube mediante el uso de las capacidades de Machine Learning locales del iPhone, aprovechando la potencia del procesador a través del framework Core ML de Apple. El objetivo es entrenar un modelo ligero especializado en visión por computadora que corra de manera local en el teléfono. Este modelo local actuará como un filtro de validación inmediata: analizará los píxeles de la cámara en milisegundos para determinar si la imagen capturada corresponde a un plato de comida válido. Si el modelo local confirmó la presencia de alimentos, se dará luz verde para consumir el ancho de banda y enviar la foto a la API avanzada en la nube; si detecta que no es comida o que la calidad es insuficiente, la aplicación detendrá el flujo localmente y notificará al usuario de inmediato, optimizando costos de operación en el servidor y mejorando la privacidad del usuario.

Con el fin de lograr la implementación de las complejas arquitecturas establecidas, y la creación de todo el software (código) que eso implica, es fundamental una buena planeación que establezca pautas para este proyecto. A continuación se presenta un consolidado del cronograma de actividades y la estimación de costos que harán posible esto.

En Bocado buscamos la eficiencia operativa y financiera es por eso que el proyecto se estructurará bajo el marco ágil Scrum, dividido en 8 Sprints de 2 semanas cada uno, asegurando una transición ordenada, a continuación se describe cada Sprint definido por el número de semanas que se estarán trabajando en el proyecto:

- Sprint 1 - 2 (Semanas 1 - 4) | Infraestructura y Backend Base: Migración a una arquitectura limpia en *FastAPI*, configuración serverless en *AWS Lambda* y estructuración relacional en *AWS RDS (PostgreSQL)*. La adopción de *AWS Lambda (Serverless)* responde a una estrategia de optimización de costos para *BitLab*.
- Sprint 3 - 4 (Semanas 5 - 8) | Integración Core de IA (MaaS): Orquestación segura de la API multimodal de *Gemini 3.0 Flash*, ingeniería de *System Prompts* para forzar respuestas en JSON estructurado y pipeline de captura con *AVFoundation*.
- Sprint 5 - 6 (Semanas 9 - 12) | Módulos Comerciales: Desarrollo del almacenamiento histórico de "Mi Diario", contador interactivo de agua y el ecosistema educativo "Aprende" con vinculación por ID único para los profesionales de la salud.
- Sprint 7 - 8 (Semanas 13 - 16) | Core ML, Seguridad y Despliegue: Integración del modelo de visión local por hardware en el iPhone (NPU), auditoría de la tríada de seguridad (*Confidencialidad, Integridad y Disponibilidad*) alineada a criterios SOC2 Tipo 2, pruebas de carga y lanzamiento en la Apple App Store.

Con esto podemos asegurar un desarrollo eficiente y que cubrirá cada punto necesario conforme a lo diseñado para la aplicación con esto podemos asegurar un software con IA híbrida (Core ML + MaaS) seguro, funcional y con bajo costo de mantenimiento.

Con los tiempos de desarrollo, buscamos también tener un modelo de eficiencia de costos, esto derivados de sus decisiones de arquitectura técnica. Al enfocar un procesamiento pesado a la infraestructura de la nube administradas y ejecutar validaciones de bajo costo de forma nativa en el dispositivo, *BitLab* logra minimizar los gastos de mantenimiento y maximizar el rendimiento de la inversión.

El costo de la dedicación de los participantes del proyecto se calcula en base a la dedicación y conocimientos en los que se enfocarán en 16 semanas (4 meses) planificadas para el desarrollo del proyecto. Los montos para cada integrante durante este periodo se muestra en la siguiente tabla.

Integrante	Duración	Costo Total
Mirna — Project Manager & UX/UI Designer	4 meses	\$128,000
Eduardo — Backend Developer & DB Manager	4 meses	\$152,000
Heber — AI Engineer & iOS Developer	4 meses	\$180,000

Para dar de alta el entorno productivo, se contemplan los siguientes gastos fijos obligatorios de licenciamiento, enrutamiento y seguridad perimetral :

- Apple Developer Program (1,800.00 MXN): Costo de la membresía anual corporativa requerida por Apple para la distribución de aplicaciones comerciales a través de la App Store
- Configuración e Infraestructura de Red Base AWS (\$540.00 MXN): Es un pago único que hacemos donde vivirá la aplicación. En lugar de comprar un servidor físico, contratamos un espacio propio, privado y seguro dentro de AWS.
- Dominio y Certificados SSL Corporativos (\$360.00 MXN): Adquisición del dominio web y firma de certificados criptográficos TLS/SSL para garantizar que toda la comunicación entre los iPhones de los usuarios y los endpoints de FastAPI viaje cifrada de extremo a extremo en tránsito.

Al haber seleccionado un modelo *PaaS* y *Serverless* para BitLab, se elimina la necesidad de invertir en costosos servidores físicos locales (*On-Premise*) o infraestructura ociosa (*IaaS*), logrando que los costos operativos se indexen de manera directa a la demanda real del mercado.

Para evaluar la salud financiera del proyecto en su etapa de tracción, se proyecta un escenario de lanzamiento estimado para una base inicial de *5,000 Usuarios Activos Mensuales (MAU)* que interactúan de forma masiva con la plataforma, ejecutando un promedio combinado de *300,000 peticiones de escaneo analítico al mes*:

- AWS Lambda & API Gateway (*Backend FastAPI Serverless* \$216.00 MXN / mes): Capa de cómputo serverless que aloja el backend en FastAPI.
- AWS RDS (*Base de Datos PostgreSQL - Instancia db.t4g.micro* \$450.00 MXN / mes): Instancia administrada de base de datos relacional encargada de la persistencia segura de perfiles de usuario.
- AWS S3 (*Almacenamiento de imágenes de comida en formato JPEG* \$144.00 MXN / mes): Almacenamiento optimizado de objetos binarios (BLOB) utilizado para resguardar de manera histórica las imágenes de los platos de comida en formato JPEG comprimido que alimentan el módulo de "Mi Diario".
- MaaS (*API de Gemini 3.0 Flash en Google AI Studio* \$810.00 MXN / mes): Costo transaccional de consumo de tokens derivado de las peticiones de análisis multimodal procesadas por la API de *Gemini 3.0 Flash* en Google AI Studio.
- Soporte de Herramientas Colectivas (*Notion, GitHub, Drive* \$270.00 MXN / mes): Licenciamiento de plataformas colaborativas integradas necesarias para el control de versiones del código fuente, documentación técnica y seguimiento continuo de tareas.

Es crucial destacar la relevancia de Core ML dentro del modelo de negocio de BitLab. Al descartar de manera local imágenes borrosas, accidentales o que no correspondan a alimentos, la aplicación mitiga el envío de tráfico basura a la nube. Esto evita llamadas innecesarias a la API de paga de Gemini y reduce el tiempo de cómputo en AWS Lambda, asegurando que el gasto operativo mensual se mantenga predecible y optimizado.

Para finalizar, BocadoApp representa más que solo una aplicación de escaneo, buscamos acercar a las personas con ayuda de la IA a soluciones accesibles. Hoy no todas las personas tienen acceso a un nutriólogo de cabecera, a poder hacer una despensa con un monto en específico, este tipo de resoluciones, nos hace democratizar la tecnología y temas a los que hoy no todos tenemos acceso, la salud. Nuestro objetivo no es reemplazar a los especialistas, sino construir un puente entre las personas, la información y tomar mejores decisiones para su bienestar. Un bocado a la vez.

Y esto es a lo que debemos de aspirar siempre, como ingenieros en computación, debemos hacer soluciones y mejoras que ayuden a nuestra sociedad, si bien no se trata de construir una nueva tecnología innovadora, también es que lo que hoy existe sea mejor, no hay que hallar el hilo negro, pero sí acercar a la gente a que tenga un acceso eficiente y cada vez más tecnológico, porque es nuestro presente y nos ayudará a facilitar muchos aspectos de nuestra vida cotidiana.

Referencias en IEEE

[1]

IBM, "SaaS," *Ibm.com*, Apr. 20, 2022. <https://www.ibm.com/mx-es/think/topics/saas>

[2]

IBM, "multi tenant," *Ibm.com*, Oct. 11, 2021. <https://www.ibm.com/mx-es/think/topics/multi-tenant>

[3]

AWS, "SOC Compliance - Amazon Web Services (AWS)," *Amazon Web Services, Inc.*, 2026. <https://aws.amazon.com/compliance/soc-faqs/>

[4]

Content Studio, "¿Qué es el cumplimiento SOC 2 tipo II?," *Everpuredata.com*, May 30, 2023. <https://www.everpuredata.com/la/knowledge/what-is-soc-2-type-ii.html> (accessed May 27, 2026).

[5]

Fortinet, "What is the CIA Triad and Why is it important? | Fortinet," *Fortinet*, 2023. <https://www.fortinet.com/lat/resources/cyberglossary/cia-triad>

[6]

AWS, "Resources," *Amazon Web Services, Inc.*, May 13, 2026. <https://aws.amazon.com/es/products/databases/resources/> (accessed May 27, 2026).

[7]

APPLE, "Apple Developer Documentation," *developer.apple.com*. <https://developer.apple.com/documentation/coreml>

[8]

apple, "Official Apple Support," *Apple.com*, 2019. <https://support.apple.com>

[9]

Apple newsroom, "El App Store global ayudó a impulsar a los desarrolladores a un nuevo nivel y generó 1.3 billones de dólares en facturaciones y ventas durante 2024," *Apple Newsroom (México)*, 2025. <https://www.apple.com/mx/newsroom/2025/06/global-app-store-helps-developers-reach-new-heights/> (accessed May 28, 2026).

[10]

R. A. Lee, "App Store Statistics 2025: Revenue, Downloads, and Market Shifts," *SQ Magazine*, May 08, 2025. <https://sqmagazine.co.uk/app-store-statistics/>

[11]

"Apple A11 Bionic: specs and benchmarks," *NanoReview.net*. <https://nanoreview.net/en/soc/apple-a11-bionic>

[12]

M. M. Serafin, "Implementando Clean Architecture en aplicaciones .NET — Parte 1," *Medium*, Oct. 04, 2021. <https://medium.com/@msmdotnet/introducci%C3%B3n-a-clean-architecture-con-net-parte-1-9db30045f2f2> (accessed May 28, 2026).

[13]

michaelstonis, "Modelo-Vista-Modelo de vista," *learn.microsoft.com*. <https://learn.microsoft.com/es-es/dotnet/architecture/maui/mvvm>

[14]

O. Blancarte, "Observer," *reactiveprogramming.io*. <https://reactiveprogramming.io/blog/es/patrones-de-diseno/observer>

[15]

Apple Inc. (2026). *App Store Review Guidelines: Section 5 - Legal (5.1.1 & 5.1.3 Data Protection & Health Frameworks)*. Apple Developer Platform. Disponible en: <https://developer.apple.com/app-store/review/guidelines/>

[16]

iomkt.domainlogic@gmail.com, "Clean Architecture: qué es, importancia y beneficios para tu empresa - Domain Logic," *Domain Logic*, May 04, 2023. <https://domainlogic.io/clean-architecture-que-es-importancia-y-beneficios-para-tu-empresa/>